

Zugriffskontrolle vertieft: GRANT, REVOKE, DENY und die Rechte-Hierarchie

Section 7 · Governance and Security · Quelle: Databricks-Dokumentation (Unity Catalog Privileges)

Ein früheres Kapitel dieser Section hat das Unity-Catalog-Sicherheitsmodell im Überblick vorgestellt. Dieses Kapitel vertieft das praktische Handwerkszeug für die Zugriffssteuerung: die Befehle `GRANT`, `REVOKE` und `DENY`, die Prinzipale, an die Rechte vergeben werden, und wie sich Rechte über die Objekthierarchie hinweg vererben.

1. Securable Objects und die Objekthierarchie

Jedes Objekt, auf das sich in Unity Catalog Rechte vergeben lassen, wird als **Securable Object** bezeichnet: der Metastore selbst, Catalogs, Schemas, Tables, Views, Volumes, Functions und weitere. Catalogs und Schemas gelten dabei zusätzlich als **Container-Objekte**: Rechte, die auf einem Container gewährt werden, wirken sich – über Vererbung – automatisch auch auf dessen Kinder-Objekte aus. Ein `USE CATALOG`-Recht auf Catalog-Ebene ist beispielsweise Voraussetzung dafür, überhaupt auf Schemas und Tabellen darunter zugreifen zu können.

2. GRANT: Rechte vergeben

Rechte werden nicht an einzelne Personen, sondern an **Prinzipale** vergeben: einzelne Nutzerkonten, Gruppen oder Service Principals (für automatisierte Zugriffe, z. B. aus einer CI/CD-Pipeline).

```
-- Leserecht auf eine einzelne Tabelle für eine Gruppe
GRANT SELECT ON TABLE main.sales.orders TO `data-analysts`;

-- Nutzungsrecht auf Schema-Ebene (Voraussetzung, um Objekte darunter überhaupt zu sehen)
GRANT USE SCHEMA ON SCHEMA main.sales TO `data-analysts`;

-- Alle Rechte auf ein Schema an einen Service Principal (z. B. für eine CI/CD-Pipeline)
GRANT ALL PRIVILEGES ON SCHEMA main.sales TO `svc-cicd-pipeline`;
```

`ALL PRIVILEGES` ist dabei ein Sonderfall: Es fasst sämtliche für das jeweilige Objekt verfügbaren Einzelrechte zusammen, statt sie einzeln aufzählen zu müssen. Jedes Objekt hat außerdem einen **Owner**, der automatisch über alle Rechte auf diesem Objekt verfügt – einschließlich des Rechts, selbst weitere Rechte an andere Prinzipale zu vergeben.

3. REVOKE: Rechte entziehen

`REVOKE` nimmt ein zuvor gewährtes Recht wieder zurück. Wichtig für die Prüfung: Der Befehl schlägt nicht fehl, selbst wenn das angegebene Recht nie explizit gewährt wurde – er ist also gefahrlos wiederholbar.

```
REVOKE SELECT ON TABLE main.sales.orders FROM `data-analysts`;
```

4. DENY: explizite Zugriffssperre

Während `REVOKE` lediglich eine zuvor erteilte Erlaubnis entfernt, setzt `DENY` eine **explizite Sperre**, die selbst dann greift, wenn derselbe Prinzipal das Recht über einen anderen Weg (z. B. Gruppenmitgliedschaft oder Vererbung von einem übergeordneten Container) eigentlich hätte. Ein `DENY` hat also grundsätzlich Vorrang vor jedem `GRANT` – dieses Zusammenspiel wird in der Prüfung gerne anhand von Szenarien mit widersprüchlichen Rechten abgefragt.

```
-- Auch wenn die Gruppe "data-analysts" über GRANT Zugriff auf das Schema hat,  
-- wird dieser einzelnen Nutzerin der Zugriff auf genau diese Tabelle explizit verwehrt.  
DENY SELECT ON TABLE main.sales.customer_pii TO `anna.example@firma.de`;
```

5. Auflösungsreihenfolge bei widersprüchlichen Rechten

Situation	Ergebnis
Kein GRANT vorhanden	Zugriff verweigert (Standard: alles ist zunächst gesperrt)
GRANT vorhanden, kein DENY	Zugriff erlaubt
GRANT und gleichzeitig DENY vorhanden (z. B. über verschiedene Gruppen)	Zugriff verweigert – DENY gewinnt immer

REVOKE nach einem GRANT	Zugriff verweigert (wie im Ausgangszustand ohne GRANT)
-------------------------	--

Ergänzt aus der offiziellen Databricks-Dokumentation: Databricks weist ausdrücklich darauf hin, dass jedes Securable Object einen Owner besitzt, der implizit über sämtliche Rechte auf diesem Objekt verfügt – inklusive der Möglichkeit, Rechte an weitere Prinzipale zu delegieren. Container-Objekte (Metastore, Catalog, Schema) spielen im Berechtigungsmodell eine besondere Rolle, da auf ihnen gewährte Rechte automatisch an alle enthaltenen Kinder-Objekte vererbt werden – ein zentraler Baustein, um große Objektmengen effizient zu verwalten, ohne jedes einzelne Objekt separat berechtigen zu müssen.

Quelle: [Manage privileges in Unity Catalog – Databricks-Dokumentation](#)